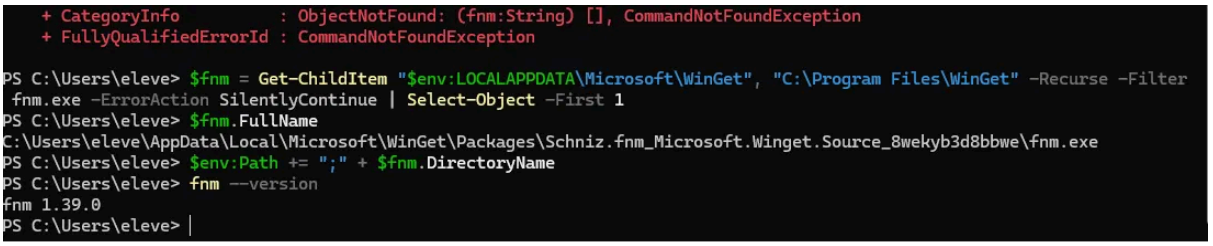
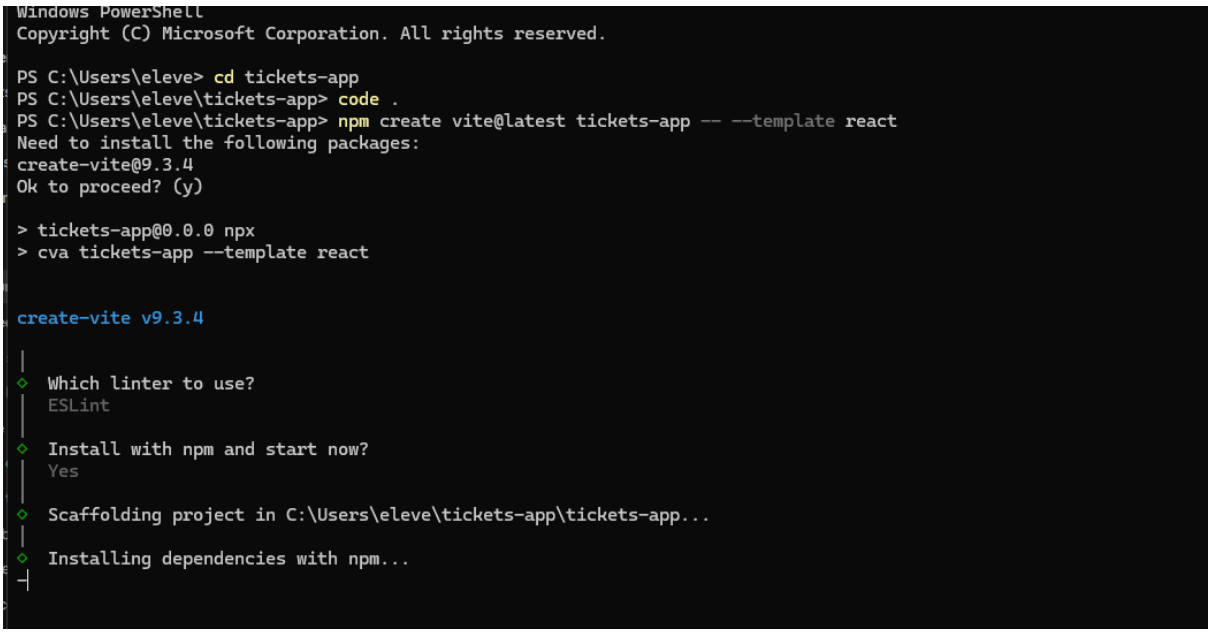
[REACT] TP1

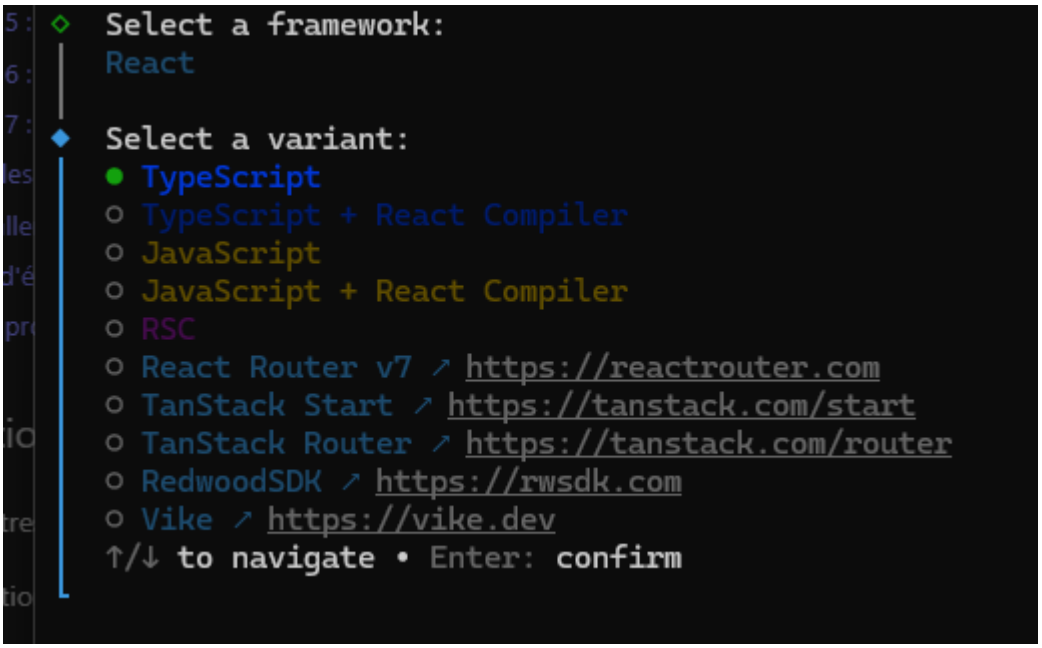
Délio Duchesne-Brajon
Groupe 1

Question 1 :
Les versions qui se sont installées sont **Node.js v26.13.0** et **npm 11.20.0**.
NPM (*Node Package Manager*) est le gestionnaire de paquets de Node.js. Il sert principalement à installer et gérer les bibliothèques dont un projet JavaScript a besoin.



Question 2 :
La première commande sert à créer un nouveau projet nommé *tickets-app* avec la dernière version de Vite, à partir du modèle React.
La seconde se place dans le dossier du projet qui vient d'être créé.
La troisième installe toutes les dépendances listées dans le fichier package.json (React, Vite, etc.) dans le dossier node_modules.
La dernière lance le serveur de développement, qui permet d'afficher l'application dans le navigateur (en général à l'adresse http://localhost:5173) et la recharge automatiquement à chaque modification.

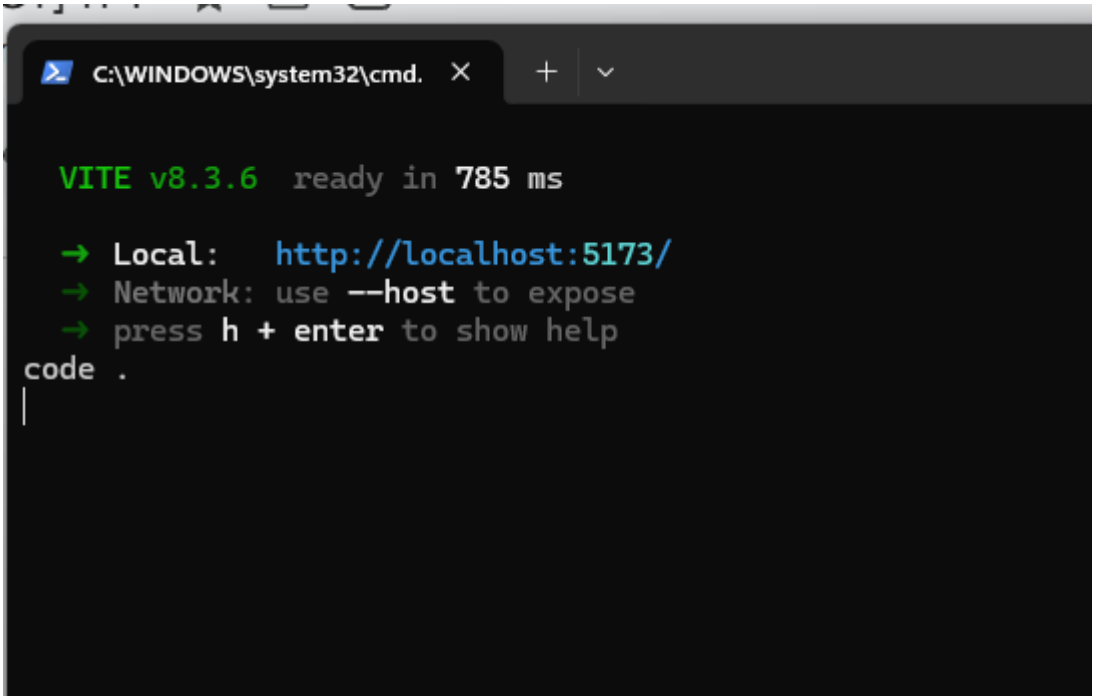


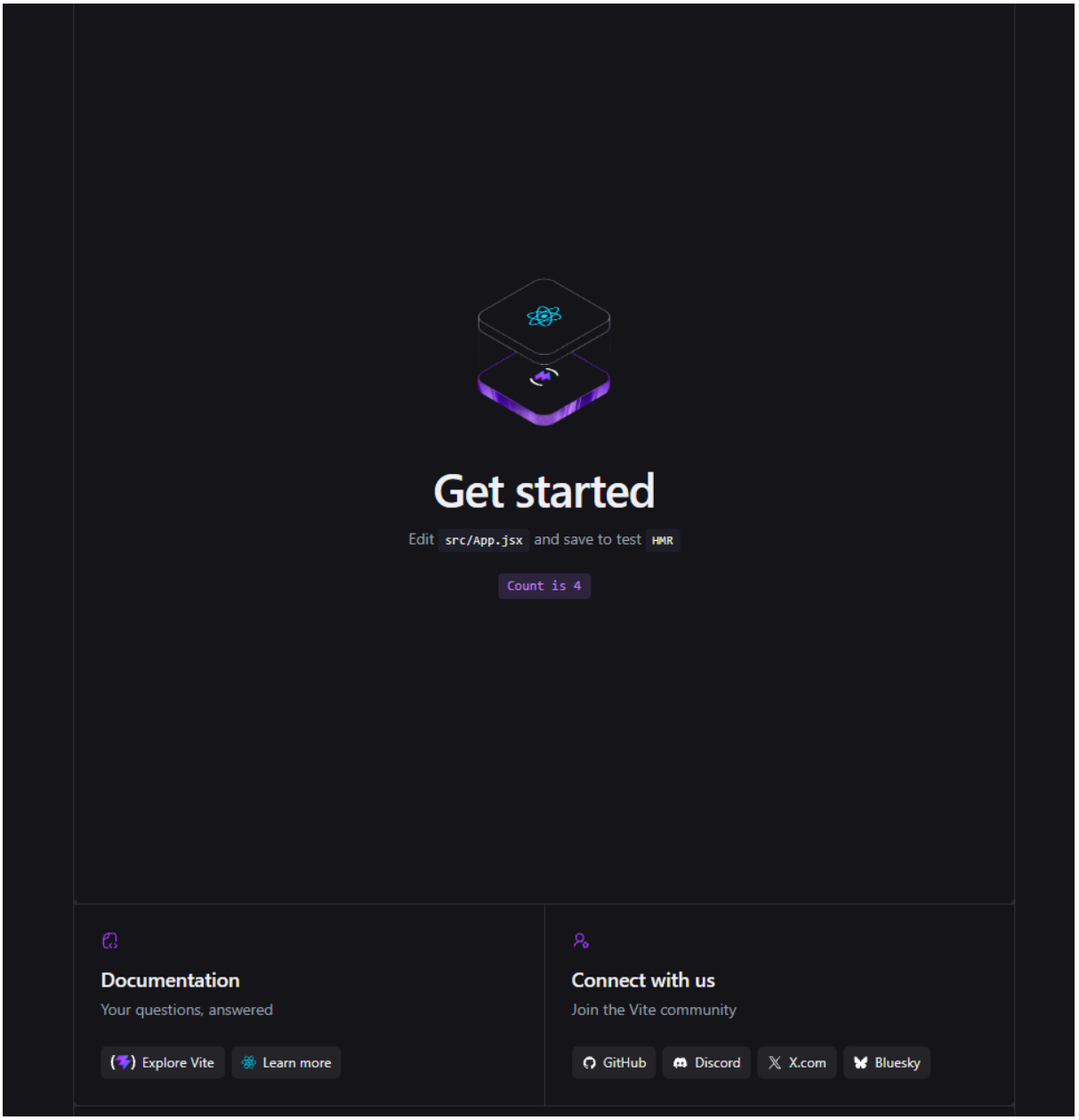


Question 3 :

Vite est un outil qui sert à développer des applications web modernes comme React. Il lance un serveur local rapide qui met à jour l'application dans le navigateur à chaque modification du code.
Contrairement à un serveur web classique comme Apache, qui envoie simplement les fichiers au navigateur, Vite transforme le code (par exemple le JSX de React) pour le rendre

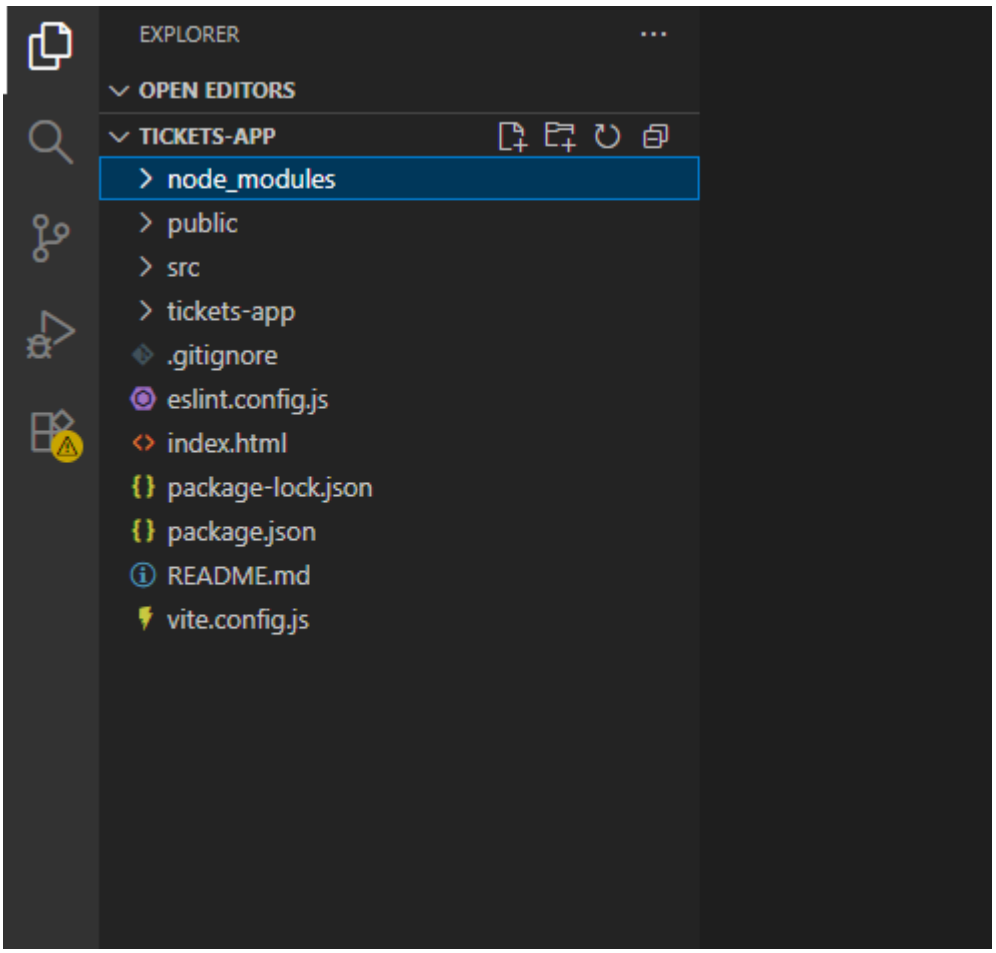
lisible par le navigateur.





Fichier / dossier	Rôle
`package.json`	Fichier de configuration du projet : il contient son nom, ses dépendances (React, Vite…) et les scripts comme npm run dev.

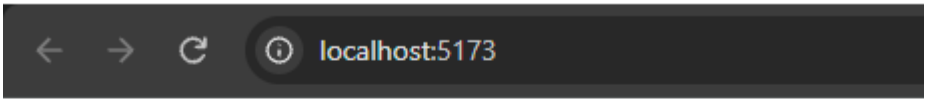
`node_modules/`	Dossier où sont installées toutes les dépendances du projet après npm install. Il n’est pas modifié à la main.
`index.html`	Page HTML principale de l’application. Elle contient la div « root » dans laquelle React affiche l’application.
`src/main.jsx`	Point d’entrée de l’application : il charge React et affiche le composant App dans la div « root ».
`src/App.jsx`	Composant principal de l’application, qui contient ce qui s’affiche à l’écran. C’est le fichier à modifier pour développer.
`src/assets/`	Dossier qui contient les ressources statiques utilisées par l’application, comme les images et les icônes.



Question 4 :

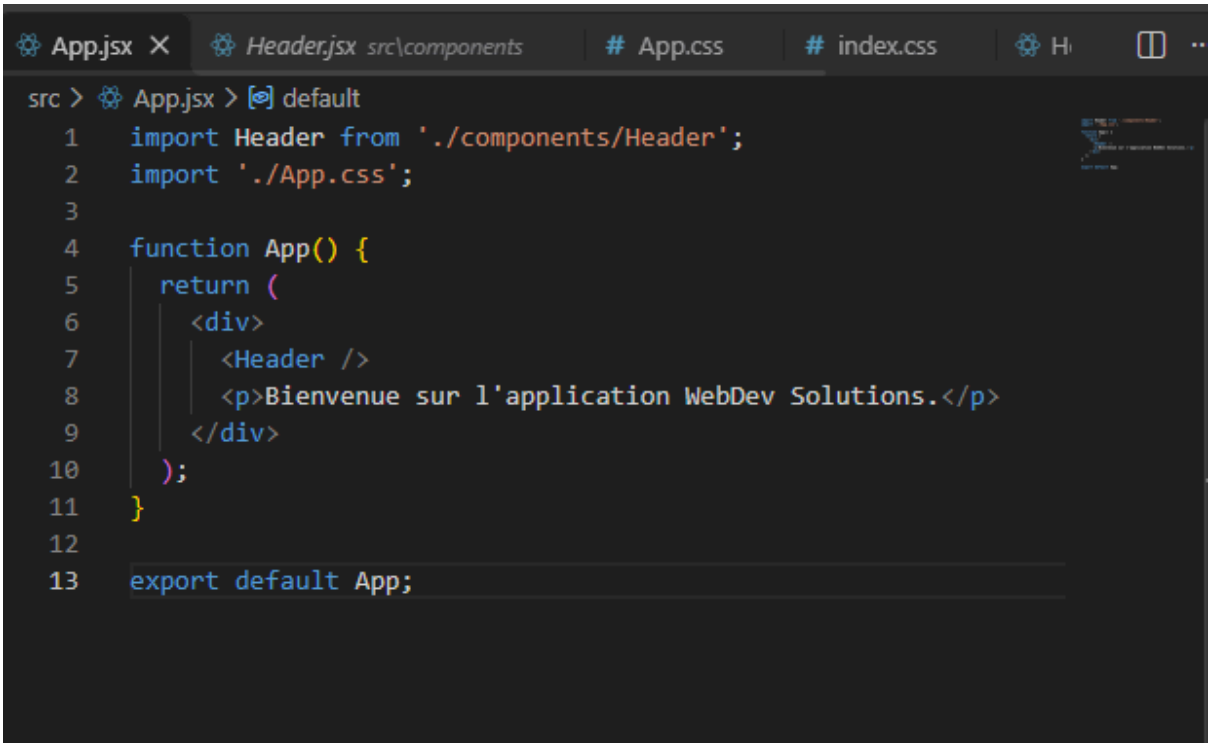
Cette ligne permet d'afficher l'application React dans la page. Elle récupère l'élément HTML ayant l'identifiant root dans le fichier index.html, créé à partir de lui une racine gérée par React, puis y affiche le composant principal <App />. Elle fait ainsi le lien entre la page HTML et le code React.

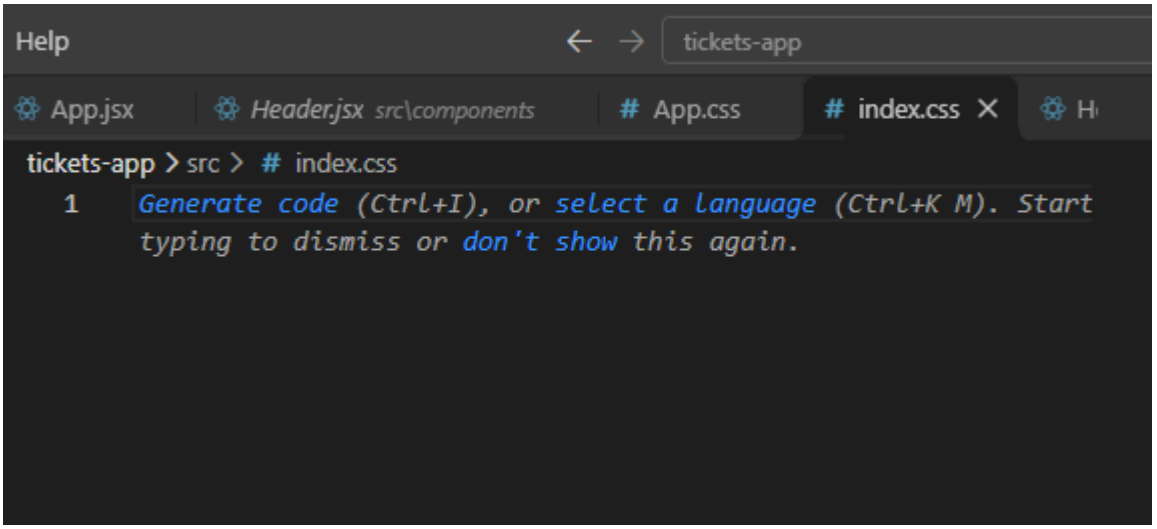
PARTIE 4 :



Gestion des tickets

Bienvenue sur l'application WebDev Solutions.





App.jsx × # App.css # index.css

c > App.jsx > default

```jsx
import { useState } from 'react'
import './App.css'

function App() {
  const [count, setCount] = useState(0)

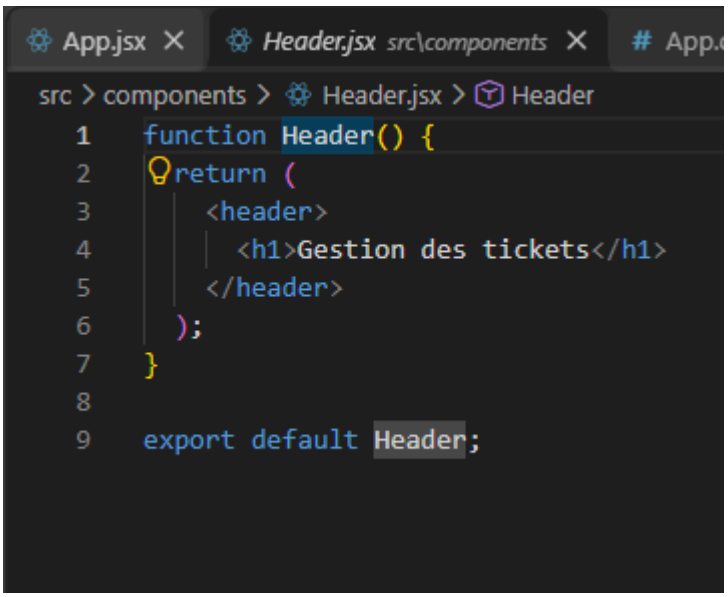
  return (
    <div className="app">
      <h1>Compteur</h1>
      <p className="count">{count}</p>
      <button onClick={() => setCount(count + 1)}>Ajouter 1</button>
    </div>
  )
}

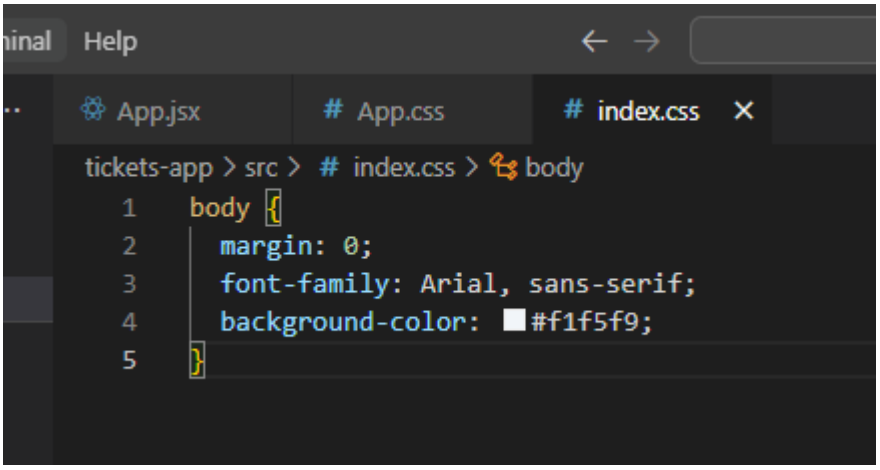
export default App
```

App.jsx × Header.jsx src\components # App.css × # index.css H

src > # App.css

```
Generate code (Ctrl+I), or select a language (Ctrl+K M). Start
typing to dismiss or don't show this again.
```





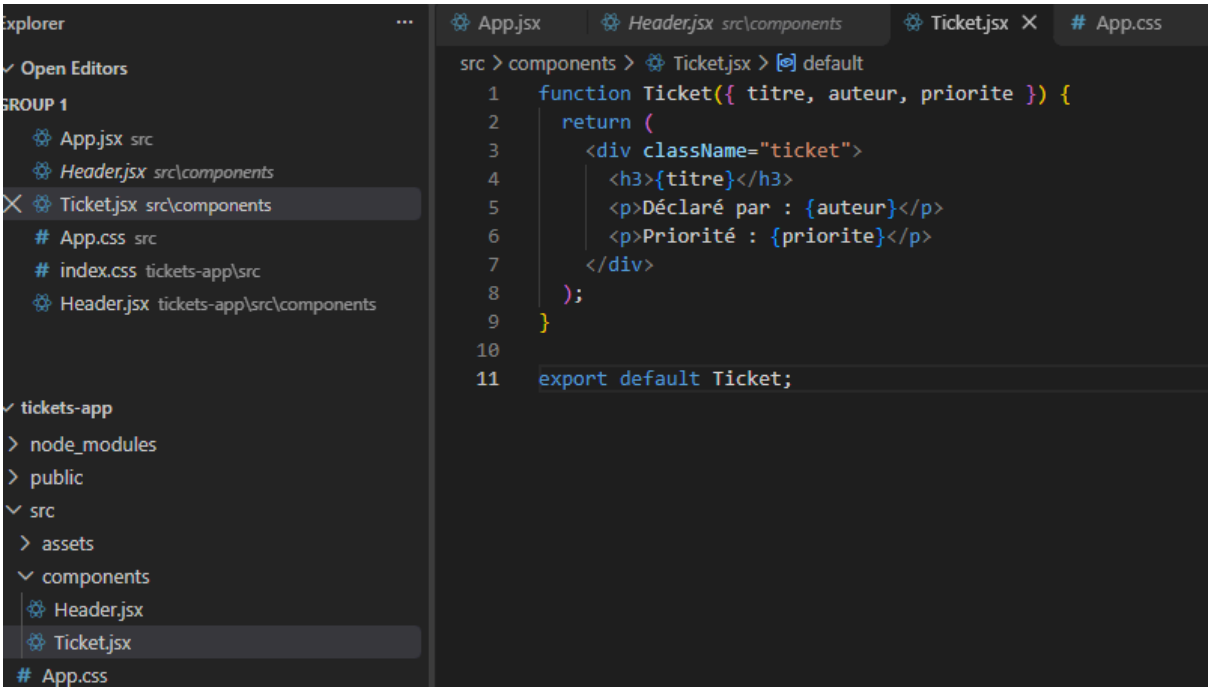
Question 5 :

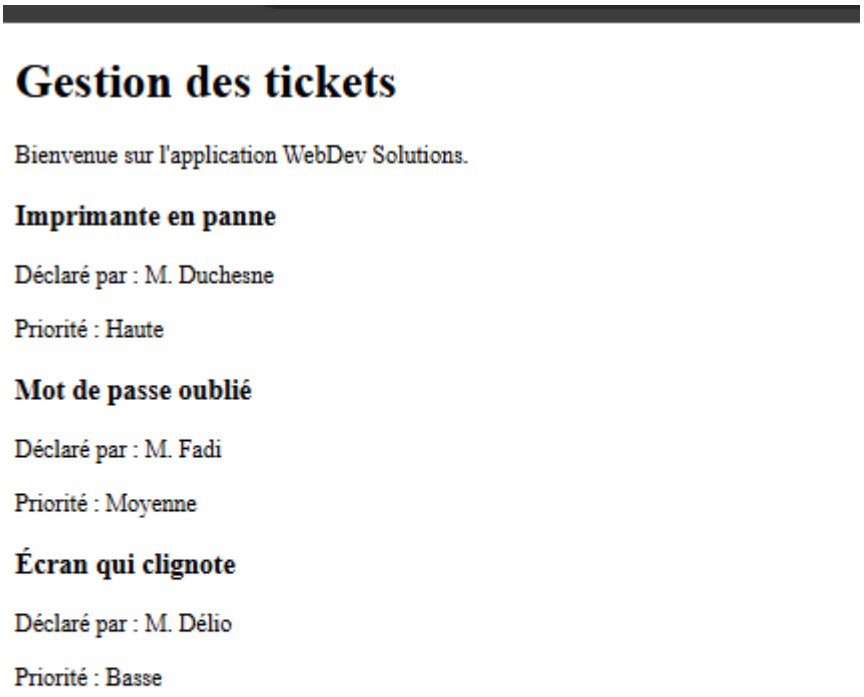
Le JSX (*JavaScript XML*) est une syntaxe qui permet d'écrire du code ressemblant à du HTML directement dans du JavaScript. React l'utilise pour décrire l'interface, et Vite le transforme ensuite en JavaScript classique compréhensible par le navigateur.

Question 6 :

Un composant doit retourner un seul élément parent car une fonction JavaScript ne peut renvoyer qu'une seule valeur. Le JSX étant transformé en appels de fonctions, plusieurs éléments côte à côte ne pourraient pas être retournés ensemble. On regroupe donc les éléments dans une balise comme une , ou dans un fragment <>...</> si l'on ne veut pas ajouter de balise supplémentaire dans la page.

Partie 5 : Les props





Gestion des tickets

Bienvenue sur l'application WebDev Solutions.

Imprimante en panne

Déclaré par : M. Duchesne

Priorité : Haute

Mot de passe oublié

Déclaré par : M. Fadi

Priorité : Moyenne

Écran qui clignote

Déclaré par : M. Délio

Priorité : Basse

Travail à faire :

App.jsx | Header.jsx src\components | Ticket.jsx | App.css | index.css | Heade

src > components > Ticket.jsx > Ticket

```jsx
function Ticket({ titre, auteur, priorite, statut }) {
  let couleur;
  if (statut === 'Ouvert') {
    couleur = 'red';
  } else if (statut === 'En cours') {
    couleur = 'orange';
  } else if (statut === 'Résolu') {
    couleur = 'green';
  }

  return (
    <div className="ticket">
      <h3>{titre}</h3>
      <p>Déclaré par : {auteur}</p>
      <p>Priorité : {priorite}</p>
      <p style={{ color: couleur }}>Statut : {statut}</p>
    </div>
  );
}

export default Ticket;
```

App.jsx | Header.jsx src\components | Ticket.jsx | App.css | index.css | Header.jsx tickets-app\...

src > App.jsx > App

```jsx
import Header from './components/Header';
import Ticket from './components/Ticket';
import './App.css';

function App() {
  return (
    <div>
      <Header />
      <p>Bienvenue sur l'application WebDev Solutions.</p>
      <Ticket titre="Imprimante en panne" auteur="M. Duchesne" priorite="Haute" statut="Ouvert" />
      <Ticket titre="Mot de passe oublié" auteur="M. Fadi" priorite="Moyenne" statut="En cours" />
      <Ticket titre="Écran qui clignote" auteur="M. Délio" priorite="Basse" statut="Résolu" />
    </div>
  );
}

export default App;
```

Question 7 :

Non, les props ne sont pas modifiables par le composant qui les reçoit : elles sont en lecture seule. Elles appartiennent au composant parent, qui les transmet à l'enfant, car les données circulent à sens unique, du parent vers l'enfant. Si l'enfant pouvait les modifier, le parent n'en saurait rien et l'affichage ne correspondrait plus à ses données. Pour changer une valeur, le parent doit donc transmettre une

nouvelle prop, et une donnée que le composant fait évoluer lui-même se gère avec un état (useState).

Partie 6 : L'état avec “useState”

```jsx
import { useState } from 'react';
import Header from './components/Header';
import Ticket from './components/Ticket';
import './App.css';

function App() {
  const [traites, setTraites] = useState(0);

  return (
    <div>
      <Header />
      <p>Bienvenue sur l'application WebDev Solutions.</p>
      <Ticket titre="Imprimante en panne" auteur="M. Duchesne" priorite="Haute" statut="Ouvert" />
      <Ticket titre="Mot de passe oublié" auteur="M. Fadi" priorite="Moyenne" statut="En cours" />
      <Ticket titre="Écran qui clignote" auteur="M. Délio" priorite="Basse" statut="Résolu" />
      <p>Tickets traités : {traites}</p>
      <button onClick={() => setTraites(traites + 1)}>
        Marquer un ticket comme traité
      </button>
    </div>
  );
}

export default App;
```

Gestion des tickets

Bienvenue sur l'application WebDev Solutions.

Imprimante en panne

Déclaré par : M. Duchesne

Priorité : Haute

Statut : Ouvert

Mot de passe oublié

Déclaré par : M. Fadi

Priorité : Moyenne

Statut : En cours

Écran qui clignote

Déclaré par : M. Délio

Priorité : Basse

Statut : Résolu

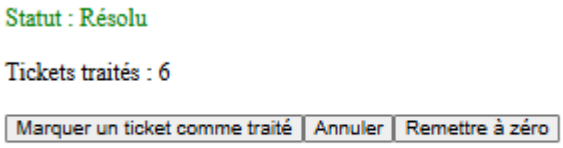
Tickets traités : 0

Marquer un ticket comme traité

Les boutons dans app.jsx :

```jsx
<button onClick={() => setTraites(traites + 1)}>
  Marquer un ticket comme traité
</button>
<button onClick={() => setTraites(Math.max(traites - 1, 0))}>
  Annuler
</button>
<button onClick={() => setTraites(0)}>
```

```
const setTraites: (value: SetStateAction<number>) => void
```



Question 8 :

Une prop est une donnée transmise par le composant parent : l'enfant la lit, mais ne la modifie pas. Le state est une donnée interne au composant, qu'il gère lui-même avec “useState”. Quand le state change, React réaffirme le composant.

Question 9 :

traites est déclaré avec const, donc traites = traites + 1 provoque une erreur. Même avec let, React ne serait pas prévenu du changement : la page ne se mettrait pas à jour et la valeur serait perdue au rendu suivant. Il faut passer par “setTraites”, qui enregistre la nouvelle valeur et déclenche un nouvel affichage.

Partie 7 : Afficher une liste

App.jsx > App

```jsx
import { useState } from 'react';
import Header from './components/Header';
import Ticket from './components/Ticket';
import './App.css';

const tickets = [
  { id: 1, titre: "Imprimante en panne", auteur: "M. Durand", priorite: "Haute", statut: "Ouvert" },
  { id: 2, titre: "Accès VPN refusé", auteur: "Mme Martin", priorite: "Moyenne", statut: "En cours" },
  { id: 3, titre: "Écran qui clignote", auteur: "M. Petit", priorite: "Basse", statut: "Résolu" },
];
```

Question 10 :

La prop key donne à chaque élément d'une liste un identifiant unique. React s'en sert pour savoir quel élément a été ajouté, modifié ou supprimé, et ne met à jour que celui-là. Si on l'oublie, la liste s'affiche quand même, mais la console du navigateur affiche un

avertissement : Each child in a list should have a unique "key" prop. React se repère alors par la position des éléments dans la liste, ce qui peut provoquer des erreurs d'affichage si la liste change.